

Causilo Technical Report

nums ai (see Appendix A for the list of contributors)

Abstract. We introduce **Causilo**, a tabular foundation model (TFM) that combines frontier predictive performance with exceptionally fast inference. On TabArena, Causilo achieves 1785.4 Elo, at a median inference time of 0.10 seconds per 1K test samples. It outperforms TabPFN-3.5-Fast with 31.6% less inference time, placing it on the performance–efficiency Pareto frontier. Causilo follows TabICL’s column-then-row architecture but introduces another row-refinement module before row compression. This module exchanges information among cell representations within each row after column encoding. The refined cells then visit the context set again through an additional column stage before being compressed into row embeddings. For inference efficiency, both row stages use cross-attention through a fixed number of summary tokens, keeping their attention cost linear in the number of features. Pretrained on approximately 36M synthetic tables, Causilo delivers strong benchmark results across TabArena, BeyondArena, and ScoringBench, achieving frontier-level performance with substantially faster inference.

DATE	18 Sep 2026
MODEL	https://huggingface.co/nums-ai/causilo
CODE	https://github.com/nums-ai/causilo

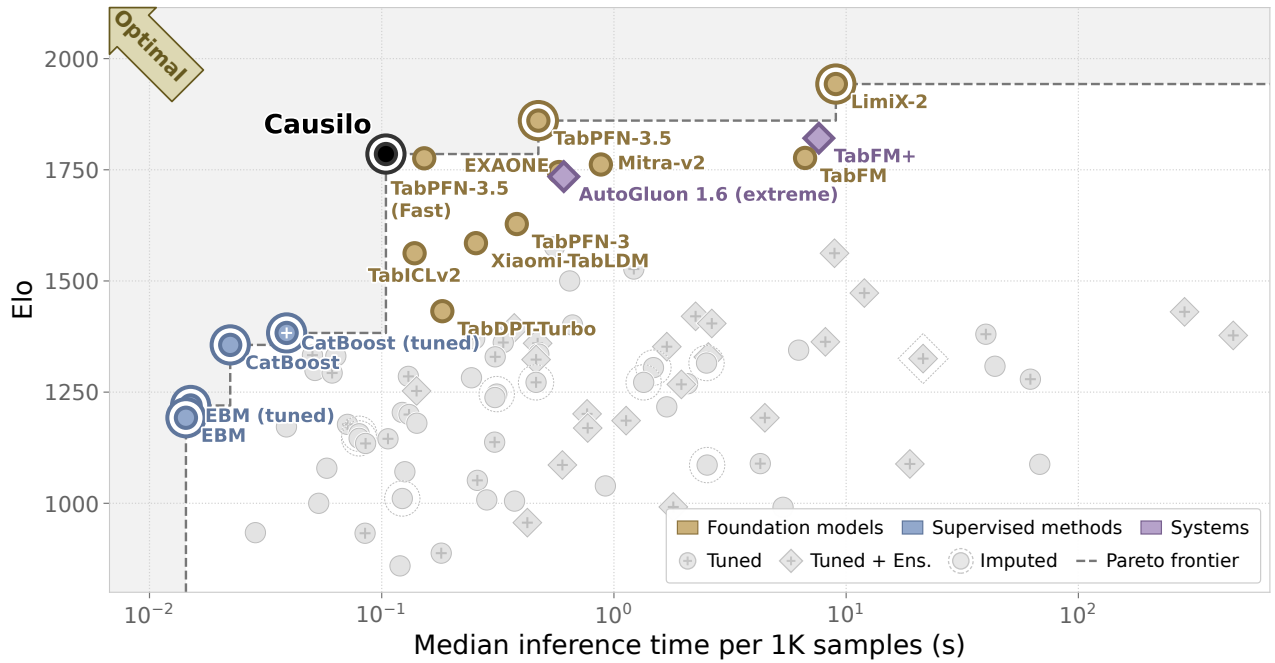


Figure 1. TabArena performance–efficiency frontier. The horizontal axis shows median inference time per 1K test samples on a logarithmic scale. Causilo achieves 1785 Elo at 0.1043 seconds, placing it on the Pareto frontier. Its classification and regression models contain 36.08M and 37.08M parameters, roughly one-sixth as many as TabPFN-3.5 (218.98M) and one-eleventh as many as LimiX-2 (406.2M), which also lie on the frontier.

1 Introduction

We introduce **Causilo**, a tabular foundation model (TFM) that combines strong predictive performance with outstandingly fast inference. Its classification and regression models contain 36.08M and 37.08M parameters, respectively. On TabArena [1], Causilo surpasses the previous state-of-the-art TabFM [2] in Elo while running faster than TabICLv2 [3], previously the fastest TFM on the Pareto frontier.

TFMs are pretrained on diverse synthetic tasks to learn a reusable prediction procedure that transfers across datasets. At inference, they condition on labeled *context* examples to predict the targets of *query* examples without updating their parameters. TabPFN [4] and TabPFN-2 [5] established this paradigm as a leading approach to tabular prediction. Modern TFMs largely use Transformer architectures [6], but differ in how they organize computation across features and samples.¹

One line of work maintains cell representations throughout the network. TabPFN-2.5 [7] and LimiX [8], for example, alternate attention across features and samples. Cell tokens exchange information within each row and then incorporate information from context rows within each column. This preserves rich representations but incurs quadratic attention costs in both the number of features and the number of samples. Recent models such as EXAONE-Tabular [9] retain this design, repeatedly updating cell tokens through attention along both the feature and sample axes.

A second line of work compresses each row into a fixed-width embedding, then uses an in-context learning (ICL) Transformer to predict query targets from labeled context rows. TabICL [10] was the first to introduce this column-then-row architecture. It first updates cell tokens across samples within each column, then compresses the resulting representations into a fixed-width embedding for each row. A separate ICL Transformer processes these row embeddings, so its attention cost no longer depends on the number of features. TabICLv2 [3] and TabPFN-3 [11] follow the same general design. This makes cross-row processing more efficient, but subsequent layers can no longer update individual cell tokens.

These architectural choices result in a clear performance–efficiency trade-off. On TabArena, column-then-row models provide low inference latency, while models that retain cross-axes attention achieve stronger Elo at greater inference cost. Dataset-specific fine-tuning, as used by Mitra-v2 [12], offers another route to stronger predictions, but requires additional optimization for every new dataset. Therefore, the resulting frontier leaves an important architectural question: *can a TFM retain richer cell representations while preserving the efficiency of in-context learning over row embeddings?*

Causilo addresses this trade-off by introducing an additional refinement module before row compression. The model first performs target-aware column encoding to construct cell representations informed by the context set. A row-refinement module then exchanges information among cell tokens within each row through a small set of summaries that persist across its layers. Rather than immediately compressing the refined cell tokens into a row embedding, Causilo sends them through a second column stage. Therefore, each cell token gets to revisit the context set after incorporating information from other features in the same row. Only afterward does a separate readout compress the cell tokens into fixed-width row embeddings for in-context prediction. This design preserves an additional round of within-row and cross-row interaction while retaining an efficient ICL backbone.

Moreover, Causilo accelerates inference by making within-row attention linear in the number of features. While existing column-then-row architectures use full self-attention among cell tokens to form row embeddings, Causilo uses cross-attention through a fixed number of summary tokens in both row refinement and compression. This replaces quadratic interactions among cell tokens with linear cost.

¹We use *features* and *columns*, as well as *samples* and *rows*, interchangeably throughout the paper. We use *cell tokens* or *cell representations* for the vectors processed before row compression, and *row embeddings* or *row representations* for the vectors produced by compression.

Pretrained entirely on approximately 36-million synthetic tables with varying sizes and causalities, Causilo demonstrates strong performance across three complementary benchmarks:

- **TabArena** [1]. Causilo achieves 1785 Elo at a median inference time of 0.1043 seconds per 1K test samples. It exceeds TabFM in Elo with 63.8 times faster inference. It lies on the empirical Pareto frontier in both classification and regression.
- **BeyondArena** [13]. Across a broader collection of datasets, Causilo ranks first among the evaluated models in both classification and regression, with Elo ratings of 1346 and 1394. Its combined Elo of 1359 leads both foundation models and conventional supervised methods.
- **ScoringBench** [14]. Causilo achieves Pareto-optimal performance across RMSE, R^2 , MAE, and CRPS with a median inference time of approximately 0.3 seconds per fold. It outperforms TabPFN-3.5-Fast on all four metrics while requiring only 40% of the inference time of TabPFN-3.5.

We release the model and inference code to support reproducible comparisons and practical use.

2 Causilo

2.1 Refine First, Compress Later

A central architectural choice in Causilo is to refine cell representations before compressing them into row embeddings. As illustrated in Figure 2, the row- and column-refinement modules retain individual cell tokens. The row-refinement module first exchanges information among cell tokens within each row. The column-refinement module then revisits the context set using these enriched representations.

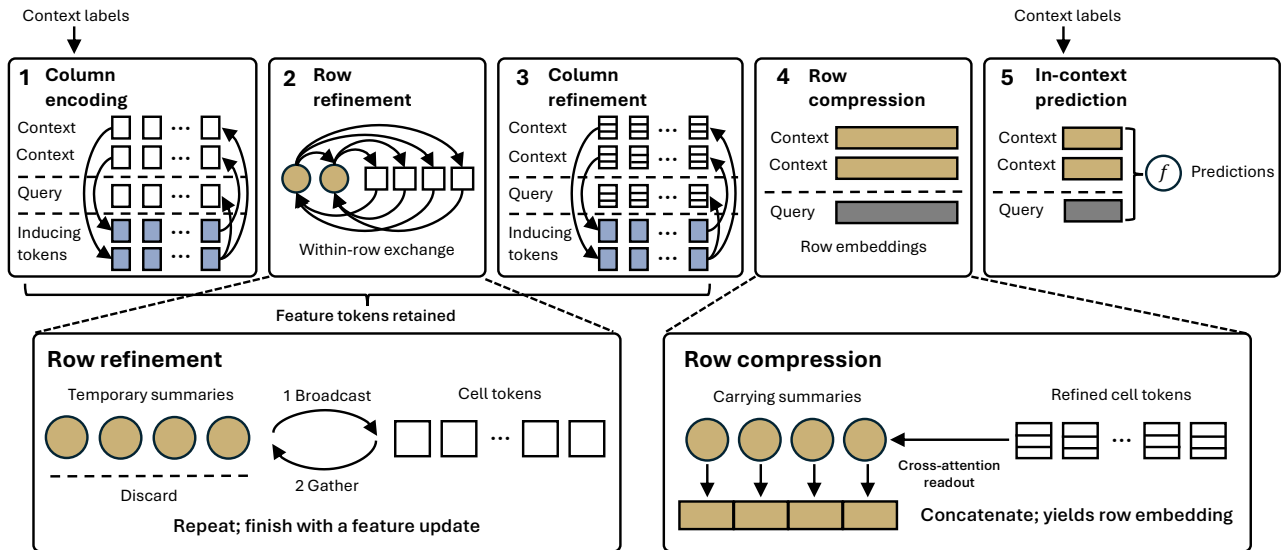


Figure 2. Architecture of Causilo. For a single input view, column encoding, row refinement, and column refinement update cell tokens while retaining the table’s row and feature-group axes. Row compression then produces a fixed-width representation for each row, followed by dataset-level in-context prediction. The lower panels expose the architectural separation at the center of Causilo: temporary summaries mediate cell representation refinement, whereas an independent set of summaries performs the final readout.

From scalar cells to representations. Causilo receives labeled context rows and unlabeled query rows. Retained features are partitioned into non-overlapping groups of three adjacent features, with zero padding for the final incomplete group. Inspired by TabFM [2], we encode observed scalars using sine and cosine features at 16 learned frequencies, followed by a shared linear projection to $\mathbb{R}^d$ with $d = 128$. Missing values use learned embeddings. Both the frequencies and missing-value embeddings

depend on the within-group position. The three cell representations in each group are summed and scaled to form a single cell token. Thus, each cell token represents one feature group in one row. For context rows, a target embedding is added to every cell token.

Column encoding. The column encoder updates cell tokens by exchanging information across rows within each column.² It uses the inducing-token attention pattern of Set Transformer [15]. Let $H_{:j} \in \mathbb{R}^{N \times d}$ denote the cell tokens in column j across N rows, and let H_{Cj} contain only the context rows. At each layer, M learned inducing tokens first *gather* information from the context cell tokens. Cell tokens in both context and query rows then read from the resulting summaries through a *broadcast*:

$$S_j = \underbrace{\mathcal{A}(U, H_{Cj}, H_{Cj})}_{\text{Gather}}, \quad H'_{:j} = \underbrace{\mathcal{A}(H_{:j}, S_j, S_j)}_{\text{Broadcast}}. \quad (1)$$

Here, $U \in \mathbb{R}^{M \times d}$ denotes the learned inducing tokens, $S_j \in \mathbb{R}^{M \times d}$ the resulting column summaries, and $\mathcal{A}(Q, K, V)$ an attention update. This enables context-to-context and context-to-query information exchange within each column. Note that query cell tokens do not contribute to the shared summaries.

Row refinement. The column encoder updates cell tokens across rows, but processes each column independently. Causilo therefore introduces within-row interactions before compression. For row i , let $H_i \in \mathbb{R}^{G \times d}$ denote its G cell tokens and $S_i \in \mathbb{R}^{K \times d}$ a set of $K = 4$ temporary summary tokens. Each refinement round alternates a *broadcast* from the summaries to the cell tokens and a *gather* of the updated cell representations back into the summaries:

$$H'_i = \underbrace{\mathcal{A}(H_i, S_i, S_i)}_{\text{Broadcast}}, \quad S'_i = \underbrace{\mathcal{A}(S_i, H'_i, H'_i)}_{\text{Gather}}. \quad (2)$$

The summary tokens carry information across refinement layers, collecting information from cell tokens and redistributing it within the row. After the final broadcast, the summaries are discarded and the refined cell tokens pass to the next stage.

Column refinement. After row refinement, each cell token carries information about other features in the same row. Causilo applies a second column stage so that these enriched representations can revisit the context set before compression. Using the row-refined cell tokens $\bar{H}$, the model again performs a context-only *gather* followed by an all-row *broadcast*:

$$\bar{S}_j = \underbrace{\bar{\mathcal{A}}(\bar{U}, \bar{H}_{Cj}, \bar{H}_{Cj})}_{\text{Gather}}, \quad H^*_{:j} = \underbrace{\bar{\mathcal{A}}(\bar{H}_{:j}, \bar{S}_j, \bar{S}_j)}_{\text{Broadcast}}. \quad (3)$$

This stage follows the same attention pattern as the first column encoder but uses independent parameters. Because its input cell tokens already contain within-row information, the second column stage can use cross-feature evidence when updating representations across rows.

Row compression. After refinement, Causilo compresses each row using a separate set of $K = 4$ learned summary tokens. Let $P_i^{(\ell)}$ denote the summary tokens at layer ℓ and H_i^* the refined cell tokens for row i . The attention update is

$$P_i^{(\ell+1)} = \mathcal{A}\left(P_i^{(\ell)}, [H_i^*; P_i^{(\ell)}], [H_i^*; P_i^{(\ell)}]\right), \quad (4)$$

where $[\cdot; \cdot]$ denotes concatenation along the token dimension. Unlike row refinement, this module updates only the summary tokens; the cell tokens remain fixed. The final summaries are individually RMS-normalized [16] and concatenated into a $Kd = 512$ -dimensional row embedding.

²After feature grouping, each column of the token grid corresponds to a group of adjacent input features. We refer to these grouped positions as columns for convenience.

Table 1. Attention complexity per row and per head for G cell tokens and K summary tokens. With fixed K , all three operations in Causilo scale linearly with G .

Operation	Complexity
Full self-attention over cell and summary tokens	$\mathcal{O}((G + K)^2)$
Causilo: one refinement round	$\mathcal{O}(K(G + K))$
Causilo: final row-refinement broadcast	$\mathcal{O}(GK)$
Causilo: one compression layer	$\mathcal{O}(K(G + K))$

In-context prediction. A separate Transformer [6] processes the row embeddings to predict query targets from the labeled context. Context targets are embedded again and added to their corresponding row embeddings. Let $U_C^{(\ell)}$ and $U_Q^{(\ell)}$ denote the context and query row representations at layer ℓ . At each layer, context representations attend to one another, while query representations attend only to the context:

$$U_C^{(\ell+1)} = \mathcal{A}(U_C^{(\ell)}, U_C^{(\ell)}, U_C^{(\ell)}), \quad U_Q^{(\ell+1)} = \mathcal{A}(U_Q^{(\ell)}, U_C^{(\ell)}, U_C^{(\ell)}). \quad (5)$$

After $L = 12$ Transformer layers, a two-layer head with GELU [17] maps each query row representation to class logits or regression outputs. Across the architecture, attention blocks use pre-RMS normalization, residual connections, and SwiGLU [18] feedforward layers.

2.2 Linear Feature Scaling by Construction

Cross-attention throughout both row stages. Existing frontier TFMs retain feature-side self-attention while individual cell representations remain explicit. Causilo instead routes within-row communication through a fixed set of summaries throughout both row modules. Refinement exchanges information in both directions between cell tokens and summary tokens; compression updates only the summaries. This asymmetry eliminates quadratic attention within rows.

Cost as the feature count grows.

Table 1 summarizes the attention cost per row and per head for G cell tokens and K summary tokens. With fixed K , attention in both row modules scales linearly with G , whereas full self-attention scales quadratically. Both column stages use a fixed number of inducing tokens, keeping their attention cost linear in the number of rows and feature. Full attention across context rows appears only in the ICL Transformer. For N_s context rows and N_q query rows, the attention cost of a forward pass is therefore $\mathcal{O}((N_s + N_q)G + N_s(N_s + N_q))$, with widths, depths, and summary counts held fixed. The first term covers cell-level column and row processing, whereas second term covers the attention in the ICL Transformer.

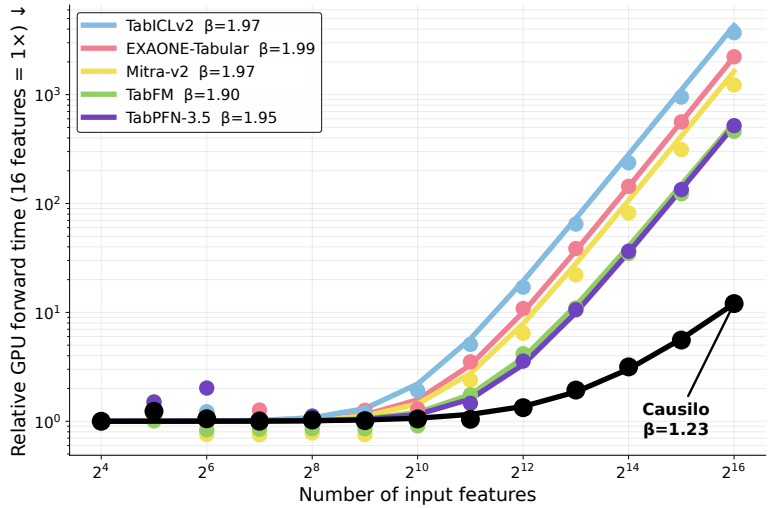


Figure 3. Feature-wise forward-time scaling of tabular foundation models. Markers show the median of 50 runs, normalized by the measured time at 16 features. Lines show fits of $T(F) = c + aF^\beta$, normalized by the fitted time at 16 features; the annotated β values are estimated over the displayed feature range.

Scaling in practice. We use a targeted GPU microbenchmark to examine whether feature-interaction time grows linearly or quadratically with feature-axis input size. For each pretrained model—Causilo, TabFM [2], Mitra-v2 [12], TabPFN-3.5 [19], TabICLv2 [3], and EXAONE-Tabular [9]—we fix the input at 16 rows and vary the feature-size parameter F from 2^4 to 2^{16} . We time the first row-mixing block of Causilo, the attention submodules of TabFM’s first row-interaction stage, and the feature-attention submodules across all layers of Mitra-v2 and EXAONE-Tabular. For TabICLv2, we time the embedding-aggregation routine within the row-interaction module; for TabPFN-3.5, we time the complete column-aggregation module. All these operations mix feature tokens within each row. After model setup, we run the selected operation 20 times for warm-up and record 50 CUDA-event timings.

Markers in Figure 3 show median timings normalized by each model’s measured median at $F = 16$. We fit the unnormalized medians to $T(F) = c + aF^\beta$, where $c \geq 0$ captures approximately fixed overhead, a sets the runtime scale, and β describes feature-dependent growth. Exponents of $\beta = 1$ and $\beta = 2$ correspond to linear and quadratic growth, respectively. The fitted curves are then normalized by their values at $F = 16$, with exponents estimated over the full displayed range. Causilo yields $\beta = 1.23$, close to linear scaling, whereas the other models yield $\beta = 1.90$ – 1.99 , close to quadratic scaling. This agrees with the expected benefit of replacing full self-attention among cell tokens within each row with cross-attention through a fixed number of summary tokens.

2.3 Bounded QASSMax

In attention over tabular data, keys can represent context rows within a column or cell tokens within a row, so context length varies with dataset size and table width. As the context grows, softmax can spread probability mass across more entries and dilute attention to relevant ones. Scalable-Softmax (SSMax) [20] addresses this effect with length-dependent scaling. TabICLv2 [3] introduces query-aware scalable softmax (QASSMax), which also adapts the scale to query content. Causilo retains this adaptivity while explicitly bounding the scaling factors.

The unbounded formulation. Let q_h denote the query vector for head h , i a coordinate within that head, and n the number of keys. TabICLv2 rescales each query coordinate as

$$\tilde{q}_{hi}^{\text{TabICLv2}} = q_{hi} \underbrace{f_{hi}(\log n)}_{\text{length-dependent scale}} \underbrace{[1 + \tanh(g_i(q_h))]}_{\text{query modulation}}. \quad (6)$$

The network f produces a length-dependent scale for each head and coordinate, while g modulates it for the current query. Although the query-dependent factor lies in $(0, 2)$, the output of f is unconstrained. The resulting multiplier can therefore be negative or arbitrarily large.

Positive and bounded scaling. Causilo retains the length-dependent network f and query-dependent network g , but bounds their outputs before applying them to the query. For an output z from either network, we use

$$\text{Bound}_L(z) = \exp \left[(\log L) \tanh \left(\frac{z}{\log L} \right) \right], \quad L > 1. \quad (7)$$

This produces a positive multiplier between $1/L$ and L , with zero mapped to one. We set $L = 8$ for f and $L = 2$ for g . To accommodate context lengths that may differ from those seen during pretraining, Causilo also introduces a smaller length-dependent network m . It provides a bounded correction to the scale produced by f , independently of query content. The complete update is

$$\tilde{q}_{hi} = q_{hi} \underbrace{\text{Bound}_8(f_{hi}(\log n))}_{\text{primary length scale}} \underbrace{\text{Bound}_2(m_{hi}(\log n))}_{\text{length correction}} \underbrace{\text{Bound}_2(g_i(q_h))}_{\text{query modulation}}. \quad (8)$$

The correction can increase or decrease the primary scale by up to a factor of two. The total multiplier remains between $1/32$ and 32 , preserving the sign of each query coordinate and preventing unbounded rescaling. The networks f and g are two-layer MLPs with 64 hidden units and GELU activations. The correction network m uses 16 hidden units and a tanh activation.

2.4 Inference-Time Ensembling

Inference-time ensembling is widely used in TFMs [9, 11, 2]. Causilo constructs ensemble members by varying feature normalization, feature order, and, for classification, class labels. All members use the same weights, and their predictions are combined after restoring the original class order or target scale.

Normalization schemes. Ensemble members cycle through four normalization schemes: standardization, rank-to-Gaussian transformation, robust scaling, and power transformation. Each scheme is fitted on the context set and followed by smooth tail compression, which reduces the magnitude of extreme values while leaving values within context-derived bounds unchanged. The same fitted transformation is applied to query rows without re-estimating its parameters.

Feature permutations. Causilo combines three adjacent features into a single token, therefore feature order determines which features are encoded together. We vary this order across ensemble members to provide different feature groupings. The first member preserves the original feature order. For the remaining members, we randomly arrange the features on a shared random ring, then generate permutations by varying the traversal stride and starting position. Candidate strides are coprime to the feature count so that each traversal visits every feature exactly once. Among these candidates, we favor strides with less frequently used one- and two-step circular distances to discourage repeated feature combinations.

Prediction aggregation. For classification, we permute class labels across ensemble members using cyclic shifts of a random class ordering. Over each complete cycle of C members, each of the C classes occupies every class index once. We restore each member’s logits to the original class order, average them across members, and apply softmax. For regression, we average the output channels of each member to obtain a point prediction. We then transform these predictions back to the original target scale and average them across members.

3 Evaluation

We evaluate Causilo on three complementary benchmarks of real-world tabular prediction. TabArena [1] assesses classification and regression performance against strong foundation and supervised models. BeyondArena [13] extends this comparison to diverse data regimes, while ScoringBench [14] evaluates both point predictions and predictive distributions for regression. We also examine inference latency on TabArena and ScoringBench to assess where Causilo lies on the performance–efficiency frontier.

3.1 TabArena

TabArena is a continuously maintained benchmark for supervised tabular learning, comprising 51 real-world datasets—38 for classification and 13 for regression. Its standardized evaluation pipeline uses shared data splits to compare tree ensembles, neural networks, and foundation models, with support for default configurations, hyperparameter tuning, and ensembling. Predictive performance is measured by ROC–AUC for binary classification, log loss for multiclass classification, and RMSE for regression. Elo ratings summarize pairwise comparisons across datasets, with higher ratings indicating stronger aggregate performance. We use the latest public results as of 18 September 2026 and Elo ratings are computed over the full leaderboard, including system submissions such as AutoGluon [21]

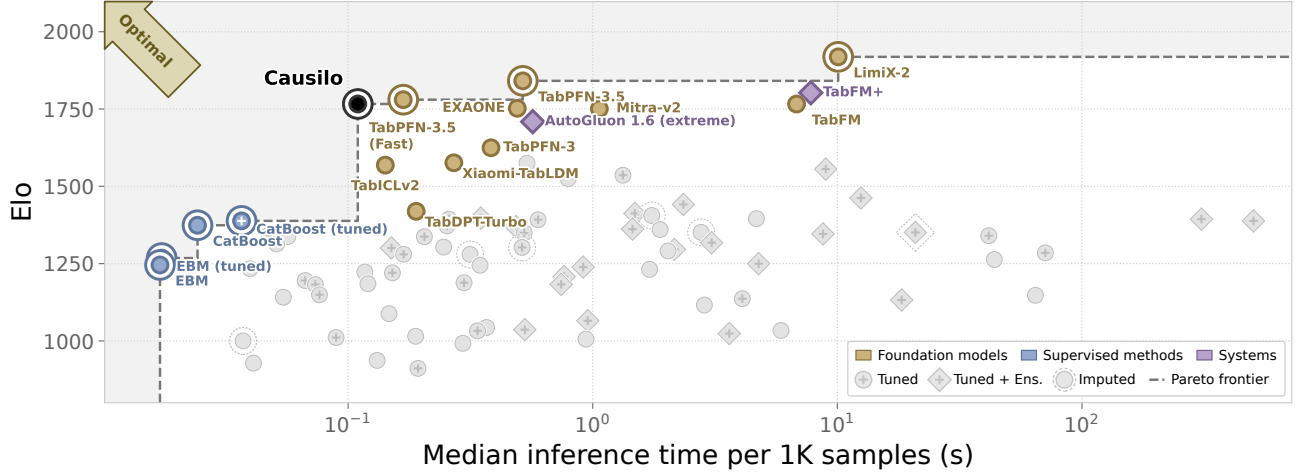


Figure 4. TabArena classification performance–efficiency frontier. Higher Elo and lower median inference time per 1K test samples are better; the time axis is logarithmic. The dashed line traces the Pareto frontier among the plotted configurations. Causilo achieves 1766.3 Elo at 0.1096 seconds per 1K test samples.

and TabFM+. A configuration lies on the Pareto frontier when no other configuration achieves both at least as high an Elo and at least as low an inference time, with a strict improvement in one dimension.

Classification. Causilo achieves 1766 Elo at 0.1096 seconds per 1K test samples (Figure 4). It essentially matches TabFM’s 1766 Elo with a 62.1-fold speedup over its 6.8094-second inference time. Relative to TabICLv2, it gains 198 Elo while requiring 22.8% less inference time. It also exceeds EXAONE-Tabular and Mitra-v2 in Elo with 4.49-fold and 9.71-fold speedups, respectively. These results place Causilo on the Pareto frontier, delivering competitive predictive performance at approximately one-tenth of a second per 1K test samples.

Higher-Elo configurations extend the frontier at increasing inference cost. TabPFN-3.5 reaches 1841 Elo at 0.5178 seconds, while LimiX-2 [22] leads with 1919 Elo but requires 10.0810 seconds, approximately 92 times Causilo’s inference time. System submissions do not improve this frontier: TabFM+ reaches 1803 Elo at 7.7865 seconds and is dominated by TabPFN-3.5.

Regression. Causilo reaches 2019 Elo at 0.0930 seconds per 1K test samples (Figure 5). Its advantage over TabFM is larger than in classification, as it gains 47 Elo while running 67.6 times faster. It also exceeds Mitra-v2 by 60 Elo with 6.41 times faster inference. Against TabICLv2, Causilo gains 350 Elo and reduces inference time by 20.1%. Thus, its low latency is accompanied by a clear improvement in aggregate predictive performance over these baselines.

Unlike in classification, Causilo dominates TabPFN-3.5-Fast, gaining 106 Elo with 22.2% less inference time. The high-Elo frontier therefore connects Causilo directly to TabPFN-3.5, which reaches 2099 Elo at 0.3765 seconds, and then to LimiX-2, which reaches 2204 Elo at 1.7817 seconds. These models require 4.05 and 19.1 times Causilo’s inference time, respectively. TabFM+ and AutoGluon 1.6 noncommercial score above Causilo, but both are dominated by TabPFN-3.5.

Combined. Figure 1 summarizes the combined comparison. Causilo achieves 1785 Elo at 0.1043 seconds per 1K test samples and occupies the low-latency end of the high-performance frontier. TabPFN-3.5 raises Elo to 1861 at 0.4727 seconds and LimiX-2 attains the highest combined Elo of 1943 at 9.0278 seconds, requiring 86.6 times Causilo’s inference time. Therefore, Causilo is a compelling choice when both predictive accuracy and fast inference are essential.

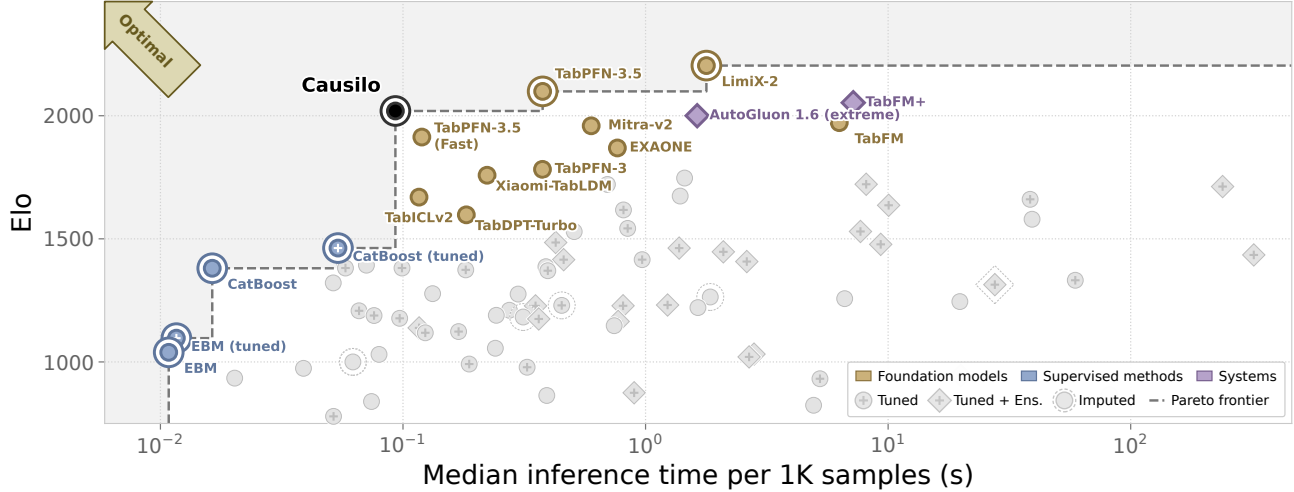


Figure 5. TabArena regression performance–efficiency frontier. Higher Elo and lower median inference time per 1K test samples are better; the time axis is logarithmic. The dashed line traces the Pareto frontier among the plotted configurations. Causilo achieves 2019.3 Elo at 0.0930 seconds per 1K test samples.

3.2 BeyondArena

BeyondArena [13] tests whether strong tabular prediction extends beyond standard IID settings. Its 142 curated datasets span IID, temporal, and grouped splits, a wide range of sample sizes and feature counts, and tables with text or high-cardinality categorical features. These settings test generalization under changes that random train–test splits can overlook. The benchmark therefore complements TabArena by covering data regimes in which conventional supervised models remain competitive. The latest public BeyondArena release includes models up to TabPFN-3. We therefore use its published baseline results and add only Causilo, which we evaluate locally.

Classification. Causilo leads the classification comparison with 1346 Elo (Figure 6a), ahead of TabPFN-3 at 1263 and RealMLP [23] at 1243 by 83 and 103 points, respectively. TabM [24], TabPFN-2.6 [25], TabICLv2, and CatBoost [26] achieve similar ratings of 1205, 1197, 1194, and 1193. Conventional supervised models remain competitive: RealMLP outperforms all evaluated TFMs except Causilo and TabPFN-3, while TabM surpasses TabPFN-2.6 and TabICLv2. Causilo leads both foundation models and conventional supervised methods across this broader collection of classification tasks.

Regression. Causilo also ranks first in regression with 1394 Elo (Figure 6b). Here, RealMLP is the closest competitor at 1357, narrowing the lead to 37 points. TabPFN-3 follows at 1294 and TabPFN-2.6 at 1265, trailing Causilo by 100 and 129 points. Causilo retains the highest aggregate rating despite the change in its closest competitor between classification and regression.

Combined. Across both tasks, Causilo achieves the highest combined Elo of 1359 (Table 2). RealMLP and TabPFN-3 are nearly tied at 1274 and 1270, leaving margins of 85 and 89 points. Causilo also attains the lowest improvability (6.4%) and average rank (6.2), together with the highest aggregated win count (47.3). Together with the task-specific rankings, these results show that Causilo’s aggregate advantage extends beyond TabArena setting and holds against both foundation and supervised models.

Classification subgroups. Causilo leads both binary and multiclass classification with 1323 and 1424 Elo, respectively (Figure 7). TabPFN-3 is the nearest competitor in both, trailing by 84 and 83 points. The nearly identical margins show that the classification advantage holds across both target types. TabPFN-3 is also the closest competitor on IID and numerical datasets, where Causilo leads by 93 points (1434 versus 1341) and 105 points (1408 versus 1303).

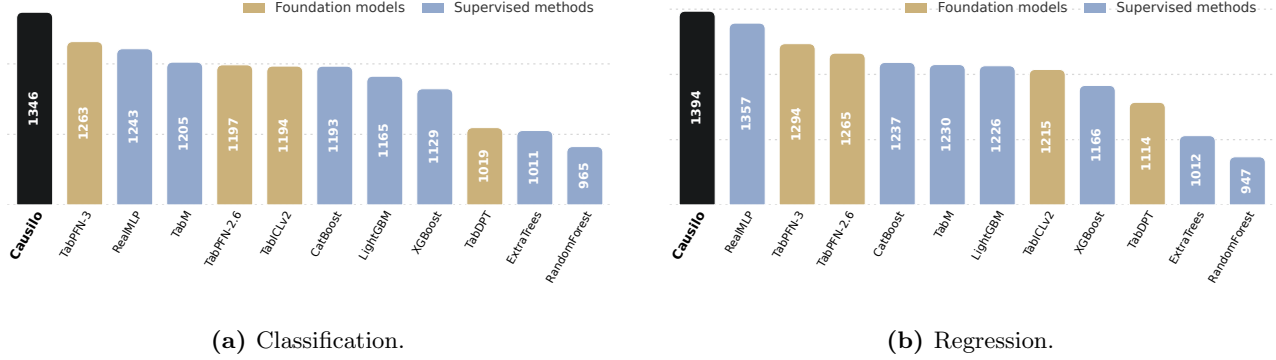


Figure 6. BeyondArena Elo of the top 12 models for classification and regression, excluding system submissions and retaining only the highest-rated variant from each model family within each comparison. Higher is better. Black denotes Causilo, gold denotes other foundation models, and blue denotes supervised methods.

Table 2. Combined BeyondArena results for 29 evaluated configurations, sorted by Elo. D denotes default, T denotes tuned, and T+E denotes tuned and ensembled configurations. Elo subscripts give the reported lower and upper confidence deviations. Wins are aggregated first-place counts and may be fractional. Best values are bold, and the Causilo row is shaded.

Model	Elo ↑	Improv- ability ↓	Avg. rank ↓	#wins ↑	Model	Elo ↑	Improv- ability ↓	Avg. rank ↓	#wins ↑
Causilo	1359 _{-40,+45}	6.4%	6.2	47.3	TabM (D)	1100 _{-27,+26}	15.5%	14.4	1.8
RealMLP (T+E)	1274 _{-26,+30}	11.9%	8.5	7.1	RealMLP (D)	1053 _{-27,+22}	17.4%	16.1	3.0
TabPFN-3 (D)	1270 _{-41,+43}	10.0%	8.6	12.5	TabDPT (D)	1047 _{-47,+37}	19.0%	16.3	5.5
TabPFN-2.6 (D)	1216 _{-37,+40}	12.4%	10.4	9.4	ExtraTrees (T+E)	1011 _{-30,+22}	19.6%	17.5	1.0
TabM (T+E)	1211 _{-27,+29}	13.2%	10.5	3.2	XGBoost (D)	1000 _{-26,+31}	19.0%	17.9	1.0
CatBoost (T+E)	1205 _{-22,+27}	13.6%	10.7	4.8	LightGBM (D)	988 _{-21,+26}	19.1%	18.3	0.8
RealMLP (T)	1202 _{-24,+31}	13.4%	10.8	6.6	ExtraTrees (T)	977 _{-34,+28}	20.3%	18.7	0.5
TabICLv2 (D)	1198 _{-41,+42}	11.9%	10.9	8.2	RandomForest (T+E)	961 _{-29,+24}	20.7%	19.3	0.9
CatBoost (T)	1189 _{-24,+26}	13.8%	11.2	5.6	RandomForest (T)	938 _{-32,+25}	21.3%	20.0	0.7
LightGBM (T+E)	1182 _{-23,+28}	14.4%	11.5	3.4	LinearModel (T+E)	880 _{-39,+30}	27.5%	21.7	2.8
TabM (T)	1171 _{-28,+27}	13.9%	11.9	3.8	RandomForest (D)	874 _{-35,+31}	23.4%	21.9	0.8
LightGBM (T)	1144 _{-23,+28}	14.9%	12.8	1.7	ExtraTrees (D)	865 _{-42,+38}	24.1%	22.2	1.2
XGBoost (T+E)	1139 _{-25,+32}	16.0%	13.0	1.8	LinearModel (T)	850 _{-45,+33}	28.3%	22.6	2.3
CatBoost (D)	1132 _{-21,+31}	15.4%	13.2	1.1	LinearModel (D)	796 _{-52,+40}	30.7%	23.9	0.5
XGBoost (T)	1117 _{-25,+29}	16.2%	13.8	2.7					

The competing models change as the feature composition changes. On low-dimensional tables, with at most 100 features after preprocessing, Causilo leads TabPFN-3 by 71 points (1403 versus 1332). On high-dimensional tables, RealMLP becomes the nearest competitor, narrowing the margin to 31 points (1248 versus 1217); TabM and CatBoost also rank above TabPFN-3. Grouped classification presents a different ordering. Across 11 datasets, Causilo is the highest-ranked TFM at 1225 Elo, behind TabM at 1283, RealMLP at 1282, and LightGBM [27] at 1228, but ahead of CatBoost at 1216.

Regression subgroups. Causilo leads on IID, numerical, and text-containing datasets, exceeding the strongest baseline in each by 147, 98, and 188 Elo, respectively (Figure 8). Its advantage is smaller across dimensionality subsets; i.e., it leads TabPFN-3 by 31 Elo on low-dimensional tables.

Grouped regression is more challenging. Across seven datasets, Causilo remains the strongest TFM at 1238 Elo but trails RealMLP by 130 points and also ranks below TabM, LightGBM, and CatBoost. These results show broad strength across data regimes, while identifying grouped generalization as a remaining gap relative to conventional supervised methods.

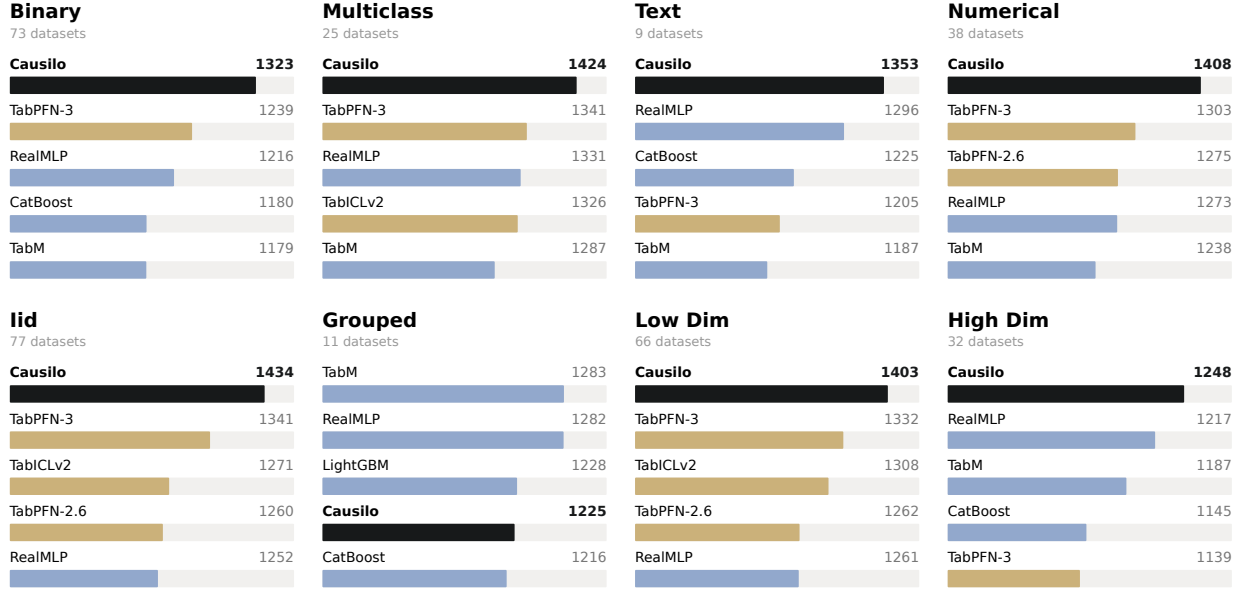


Figure 7. BeyondArena classification Elo by subgroup. Each panel shows the top five model families, retaining the highest-rated variant within each family. Dataset counts appear below the subgroup names. Black denotes Causilo, gold other TFMs, and blue conventional supervised models. Higher Elo is better.

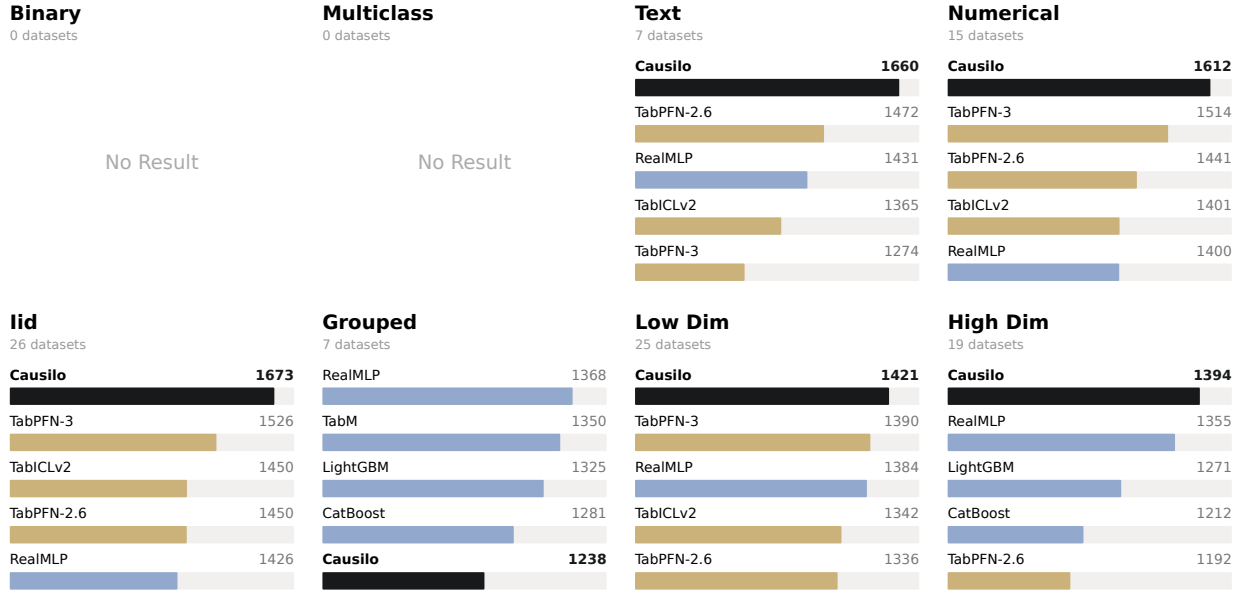


Figure 8. BeyondArena regression Elo by subgroup, using the same model-family selection and colors as Figure 7. Causilo leads five of the six displayed subgroups and is the highest-ranked TFM on grouped regression. Higher Elo is better.

3.3 ScoringBench

ScoringBench [14] evaluates both point predictions and predictive distributions on a heterogeneous collection of real-world regression datasets. Figure 9 compares metric-specific Elo ratings for RMSE, R^2 ,

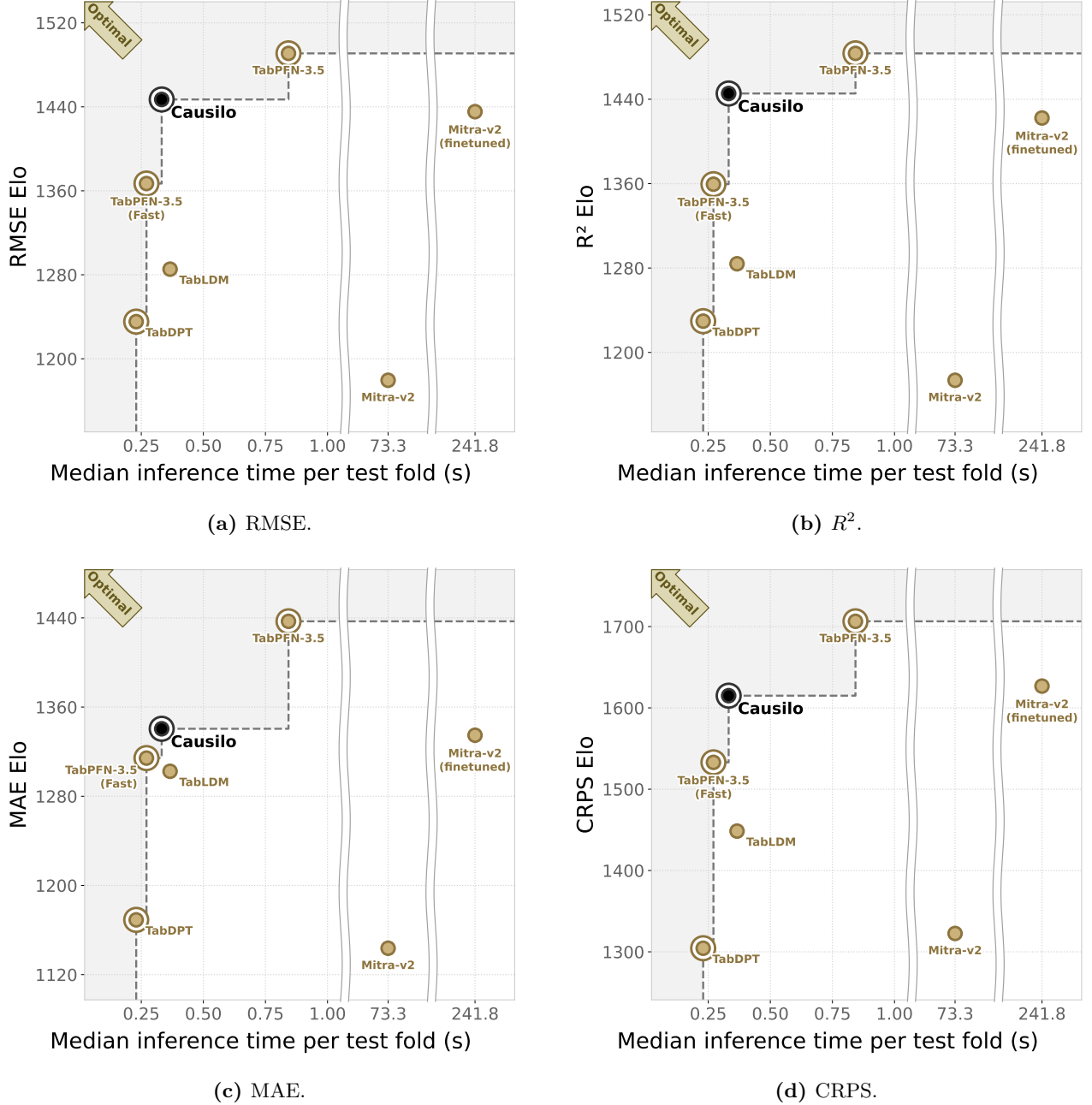


Figure 9. ScoringBench performance–efficiency frontiers for point prediction (RMSE, R^2 , and MAE) and distributional prediction (CRPS). Each panel plots metric-specific Elo against median inference time per test fold in seconds. Dashed lines trace the empirical Pareto frontier; breaks in the time axis separate the much slower Mitra-v2 configurations. Causilo lies on the frontier in all four panels.

MAE, and CRPS against median inference time per test fold. Higher Elo indicates stronger predictive performance for every metric, while lower inference time is better. The broken time axis accommodates the substantially longer runtimes of the Mitra-v2 configurations.

Point prediction. The three point-prediction metrics yield similar performance–efficiency frontiers. Causilo achieves higher Elo than TabPFN-3.5-Fast with a modest increase in inference time, while TabPFN-3.5 provides the highest Elo at a higher latency. Causilo also outperforms fine-tuned Mitra-v2

on all three point metrics at substantially lower inference cost. In the MAE panel, it additionally improves on TabLDM at lower latency, extending its advantage beyond squared-error metrics. It therefore occupies an intermediate point on the frontier between TabPFN-3.5-Fast and TabPFN-3.5, improving point-prediction quality without requiring the latency of the latter.

Distributional prediction. CRPS evaluates the full predictive distribution by integrating the squared difference between its cumulative distribution function and the step function at the observed target. It rewards accurate distributions that are concentrated without being overconfident. Lower raw CRPS is better, corresponding to higher CRPS Elo in the figure.

Causilo remains on the CRPS frontier, outperforming TabPFN-3.5-Fast while requiring only modestly more inference time. TabPFN-3.5 again attains a higher Elo at a higher latency. Fine-tuned Mitra-v2 slightly exceeds Causilo in CRPS Elo, but requires a median 241.8 seconds per test fold, compared with approximately 0.3 seconds for Causilo. Across all four metrics, Causilo combines sub-second inference with competitive point and distributional prediction quality.

4 License

The Causilo code is released under the Apache License 2.0. The pretrained model weights are distributed separately under the Causilo License v1.0, which permits non-commercial research, testing, evaluation, experimentation, and modification.

Commercial or production use of the model, its derivatives, or its outputs requires a separate license from Nums AI Inc. and any other relevant rights holders. Providing the model or its derivatives through hosted, managed, API, or SaaS services also requires a license, whether the service is paid or free.

Scholarly publication and the sharing of research results, including outputs generated in compliance with the license, are permitted. Independently authored papers and outputs need not adopt the model license; any included model components remain subject to its redistribution requirements. The full license texts govern all use and distribution. Licensing inquiries may be directed to contact@nums.world.

5 Conclusion

We introduced Causilo, a tabular foundation model that combines strong predictive performance with fast inference. By introducing an additional refinement stage before row compression, Causilo preserves richer feature-level interactions while keeping computation efficient. Across TabArena, BeyondArena, and ScoringBench, Causilo establishes a strong position on the performance–efficiency frontier.

References

- [1] Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Prateek Desai, David Salinas, and Frank Hutter. Tabarena: A living benchmark for machine learning on tabular data. *Advances in Neural Information Processing Systems*, 38, 2026.
- [2] Weihao Kong and Abhimanyu Das. Introducing TabFM: A zero-shot foundation model for tabular data. Google Research, 2026. Official release and implementation. <https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/>; <https://github.com/google-research/tabfm>.
- [3] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. Tabicl v2: A better, faster, scalable, and open tabular foundation model. *arXiv preprint arXiv:2602.11139*, 2026.
- [4] Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. *arXiv preprint arXiv:2207.01848*, 2022.
- [5] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirmer, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [7] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Benjamin Jäger, Dominik Safaric, Simone Alessi, Adrian Hayler, et al. TabPFN-2.5: Advancing the state of the art in tabular foundation models. *arXiv preprint arXiv:2511.08667*, 2025.
- [8] Xingxuan Zhang, Gang Ren, Han Yu, Hao Yuan, Hui Wang, Jiansheng Li, Jiayun Wu, Lang Mo, Li Mao, Mingchao Hao, et al. Limix: Unleashing structured-data modeling capability for generalist intelligence. *arXiv preprint arXiv:2509.03505*, 2025.
- [9] Moonjung Eo, Min-Kook Suh, Hye-Seung Cho, Jiwon Kim, Seoyoon Kim, Sangjun Nam, and Soonyoung Lee. Exaone tabular 1.0: Technical report. *arXiv preprint arXiv:2608.25774*, 2026.
- [10] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. Tabicl: A tabular foundation model for in-context learning on large data. *arXiv preprint arXiv:2502.05564*, 2025.
- [11] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Mihir Manium, Shi Bin Hoo, Magnus Bühler, Anurag Garg, et al. TabPFN-3: Technical report. *arXiv preprint arXiv:2605.13986*, 2026.
- [12] Yefan Tao, Xiyuan Zhang, Xinyi Liu, Boran Han, Danielle Maddix, Haoyang Fang, Zhen Han, Jiading Gai, Xuanqing Liu, Michael Bohlke-Schneider, et al. Mitra-v2 technical report. *arXiv preprint arXiv:2609.04540*, 2026.
- [13] Lennart Purucker, Andrej Tschalzev, Nick Erickson, Gioia Blayer, David Holzmüller, Alan Arazi, Alexander Pfefferle, Mustafa Tajjar, Gaël Varoquaux, Frank Hutter, et al. Beyond iid: How general are tabular foundation models, really? *arXiv preprint arXiv:2606.30410*, 2026.
- [14] Jonas Landsgesell, Pascal Knoll, and Tizian Wenzel. Scoringbench: A benchmark for evaluating tabular foundation models with proper scoring rules. *arXiv preprint arXiv:2603.29928*, 2026.

- [15] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pages 3744–3753. PMLR, 2019.
- [16] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. <https://arxiv.org/abs/1910.07467>.
- [17] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [18] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [19] Prior Labs Team. TabPFN-3.5: Technical report, 2026. September 14, 2026. <https://storage.googleapis.com/prior-labs-tabpfn-public/reports/tabpfn-v3.5-report.pdf>.
- [20] Ken M Nakanishi. Scalable-softmax is superior for attention. *arXiv preprint arXiv:2501.19399*, 2025.
- [21] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. Autogluon-tabular: Robust and accurate automl for structured data. *arXiv preprint arXiv:2003.06505*, 2020.
- [22] Xingxuan Zhang, Gang Ren, Hao Yuan, Hao Zou, Hongze Tan, Hui Wang, Jianhao Song, Jiansheng Li, Jiayao Zhang, Jinghan Zhang, Kaifang Li, Lang Mo, Li Mao, Mingchao Hao, Nuo Xu, Rui Ding, Ruiji Zhang, Shuyang Li, Siyu Mei, Tianyang Zhang, Weiyang Mu, Yancheng Dong, Yongxian Wei, Yuan Xue, Yuanrui Wang, Yue He, Zijia Yang, Ziyun Li, Dongzhe Li, Fuqiang Wang, Jiandong Liu, Jiawei Chen, Jiaxin Du, Kaijie Cheng, Kehan Li, Lei Sun, Linjun Zhou, Ningbo Dai, Qi Wang, Renzhe Xu, Shaoxing Du, Shumeng Yang, Wang Lu, Wenjing Chu, Xiannan Huang, Xiaoyu Lin, Xing Ai, Xinyan Han, Xuanyue Li, Xuanyue Su, Xukun Zhang, Yan Lu, Yaxin Zhang, Yi Qin, Yifei Huang, Yihan Xu, Yongle Lv, Yuanyuan Jiang, Yushan Han, and Peng Cui. Limix-2: A contextual mechanism network towards general structured-data intelligence, 2026.
- [23] David Holzmüller, Léo Grinsztajn, and Ingo Steinwart. Better by default: Strong pre-tuned mlps and boosted trees on tabular data. In *Neural Information Processing Systems*, 2024.
- [24] Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. Tabm: Advancing tabular deep learning with parameter-efficient ensembling. In *International Conference on Learning Representations*, volume 2025, pages 77899–77935, 2025.
- [25] Prior Labs. TabPFN-2.6: Model documentation and official implementation, 2026. <https://docs.priorlabs.ai/models>; https://github.com/PriorLabs/TabPFN/blob/main/src/tabpfn/architectures/tabpfn_v2_6.py.
- [26] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. *Advances in neural information processing systems*, 31, 2018.
- [27] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.

A Contributors

Minyong Cho, Minho Jeong, Dooho Lee, Jinmo Lee, and Jaemin Yoo.

The listing of contributors is in alphabetical order based on their last names.

B Acknowledgements

We thank Prior Labs for their contributions to tabular foundation models and continued efforts to advance the field. We are grateful to the TabICL authors for openly sharing their code, models, and technical insights, which informed the development of Causilo. We also thank the authors and maintainers of TabArena, BeyondArena, and ScoringBench for creating and operating benchmarks that enable rigorous evaluation and meaningful comparisons across tabular learning methods.